Chapter 3

List

Linear list concepts

Linear list concepts

Definition

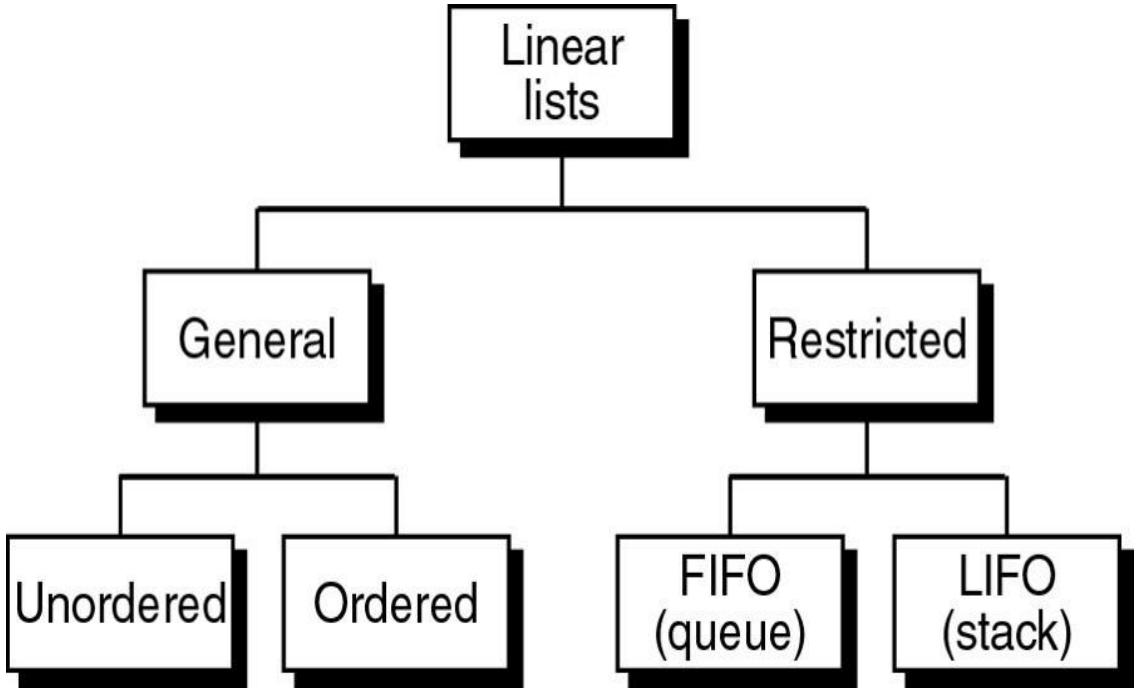
A linear list is a data structure in which each element has a unique successor.



Example

- **Array**
- **Linked list**

Linear list concepts



Linear list concepts

General list:

•A general linear list is a list in which operations, such as retrievals, insertions, changes, and deletions, can be done anywhere in the list

•Unordered list (random list): Data are not in particular order.

•Ordered list: data are arranged according to a key.

Linear list concepts

Restricted list:

- Only some operations can be used on the list.
- Data can be inserted/deleted only at the ends of the list.

- Queue: FIFO (First-In-First-Out).
- Stack: LIFO (Last-In-First-Out).

Basic Operations

We begin with a discussion of the basic list operations. Each operation is developed using before and after figures to show the changes.

- Insertion
- Deletion
- Retrieval
- Traversal

insertion

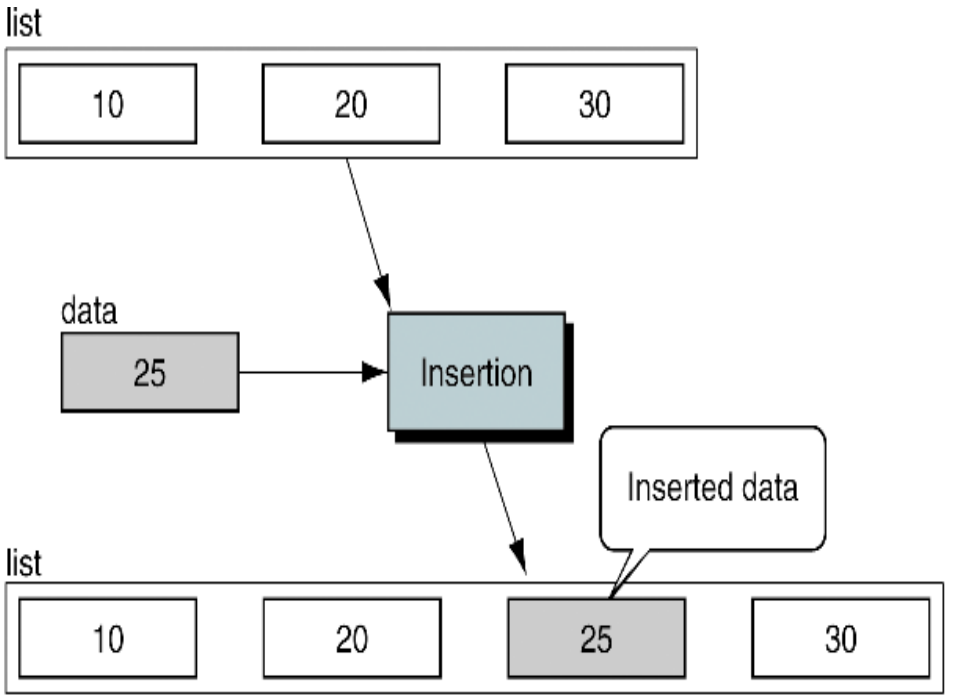


FIGURE 5-1 Insertion

Insertion is used to add a new element to the list

deletion

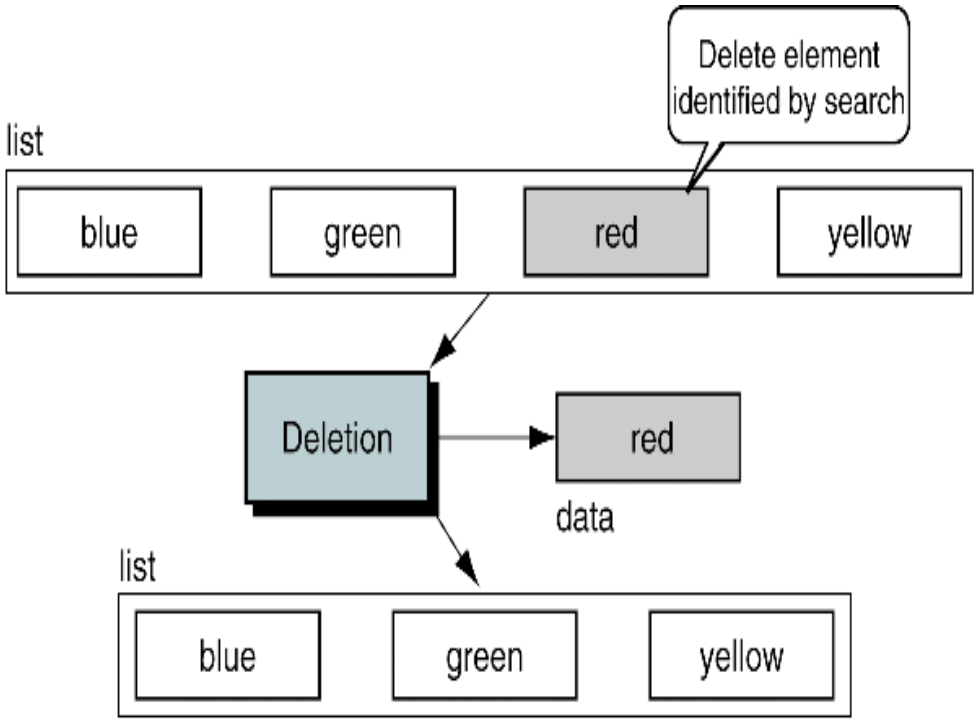


FIGURE 5-2 Deletion

Deletion is used to remove an element from the list.

retrieval

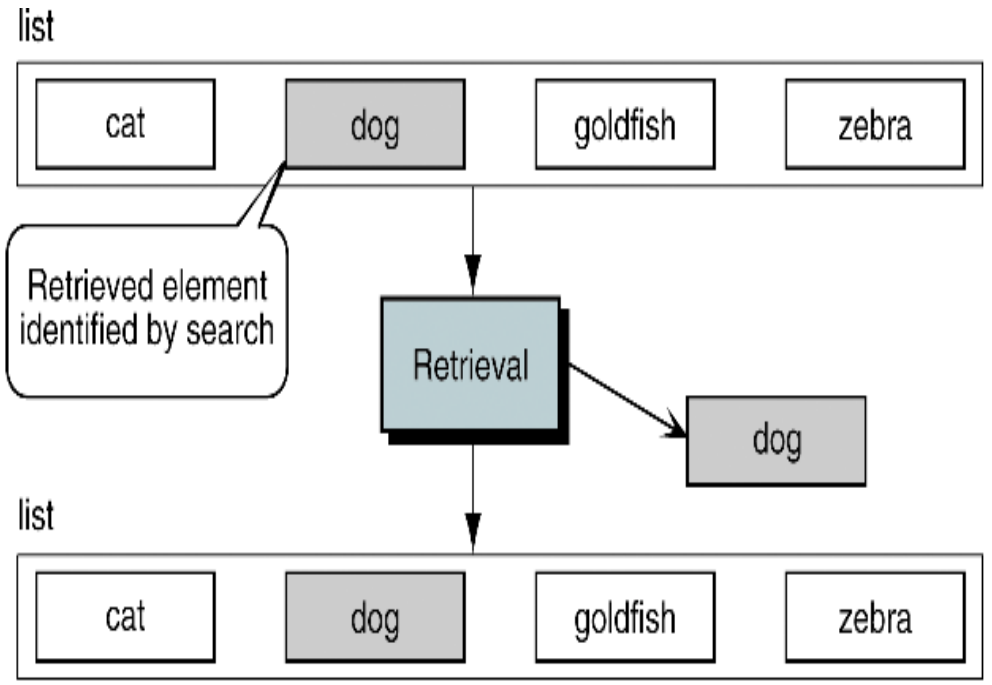


FIGURE 5-3 Retrieval

Retrieval is used to get the information related to an element without changing the structure of the list.

traversal

- List traversal processes each element in a list in sequence.

Implementation

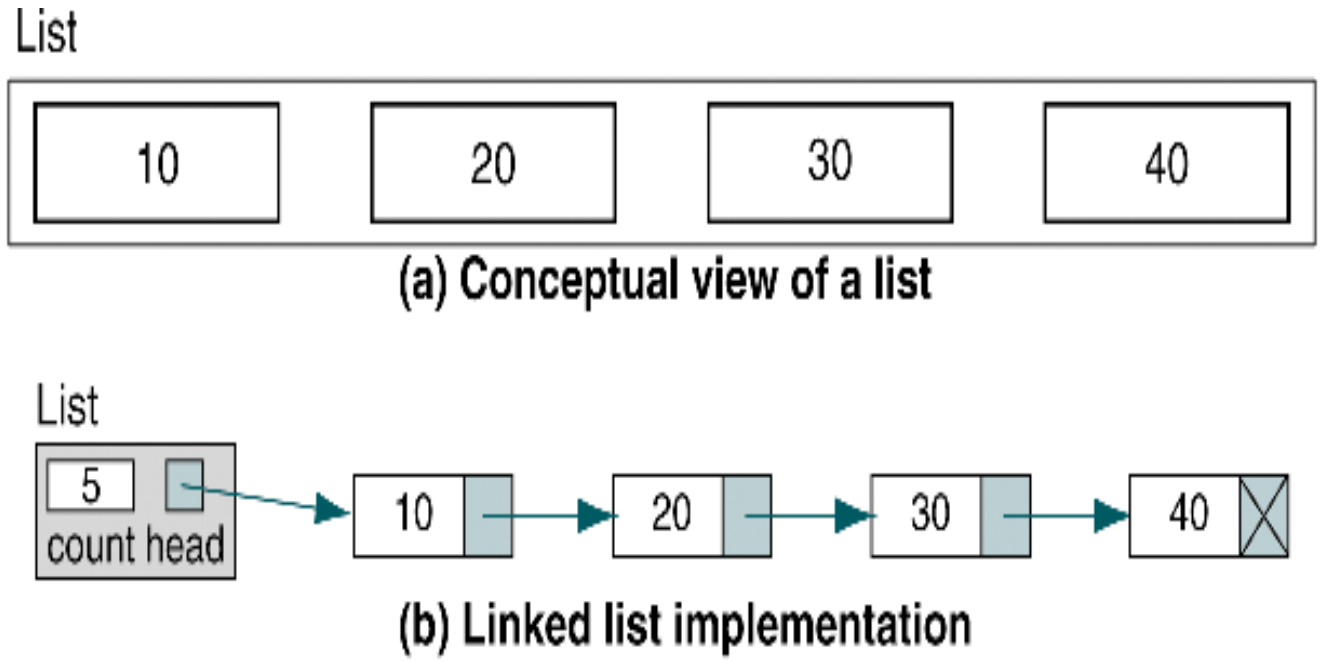


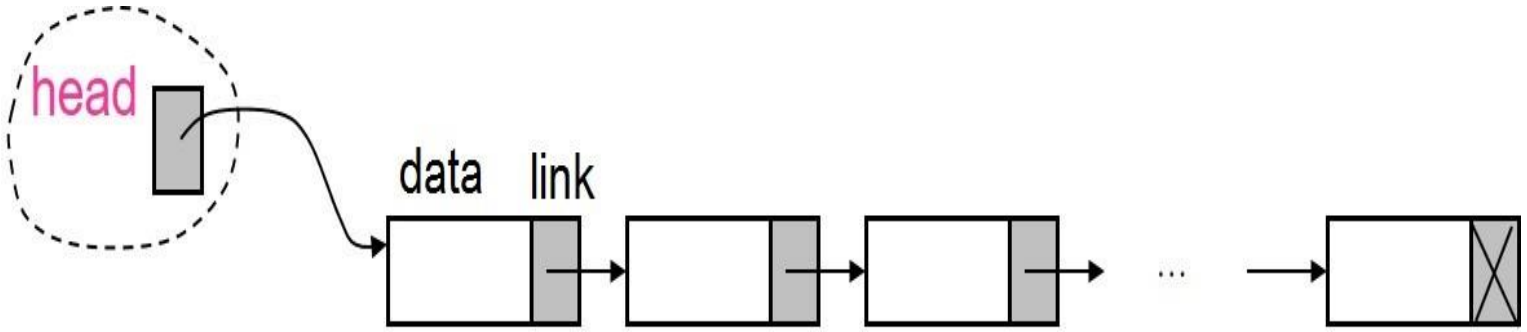
FIGURE 5-4 Linked List Implementation of a List

Singly linked list

Linked List

Definition

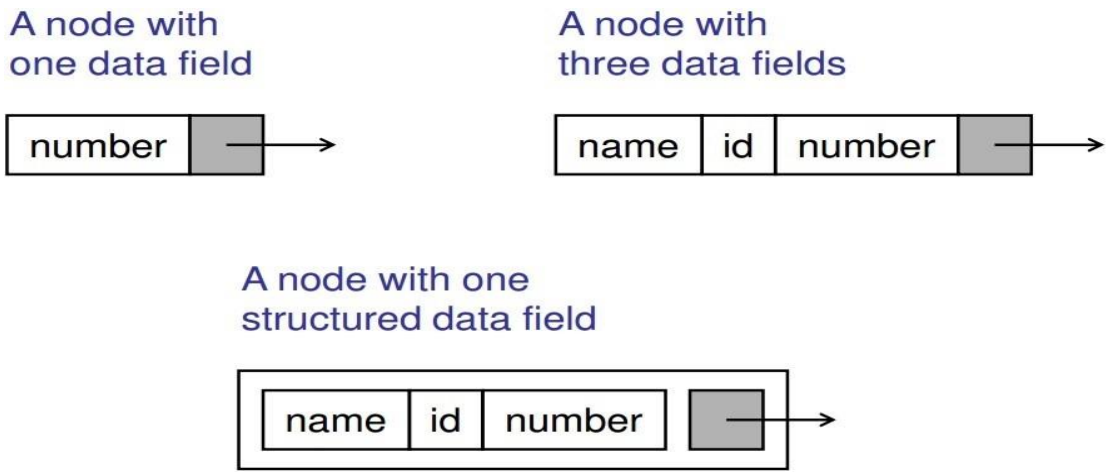
A linked list is an ordered collection of data in which each element contains the location of the next element.



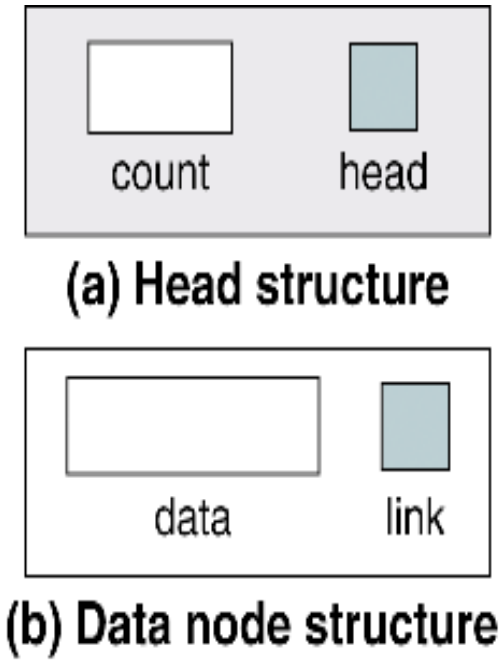
Nodes

The elements in a linked list are called nodes.
A node in a linked list is a structure that has at least two fields:

- the data,
- the reference to the next node.



Data structure




```
list
  count
  head
end list

node
  data
  link
end node
```

FIGURE 5-5 Head Node and Data Node

Algorithms

- Create list
- Insert node
- Delete node
- List search
- Retrieve node
- Empty list
- Full list
- List count
- Traverse list
- Destroy list

Create list

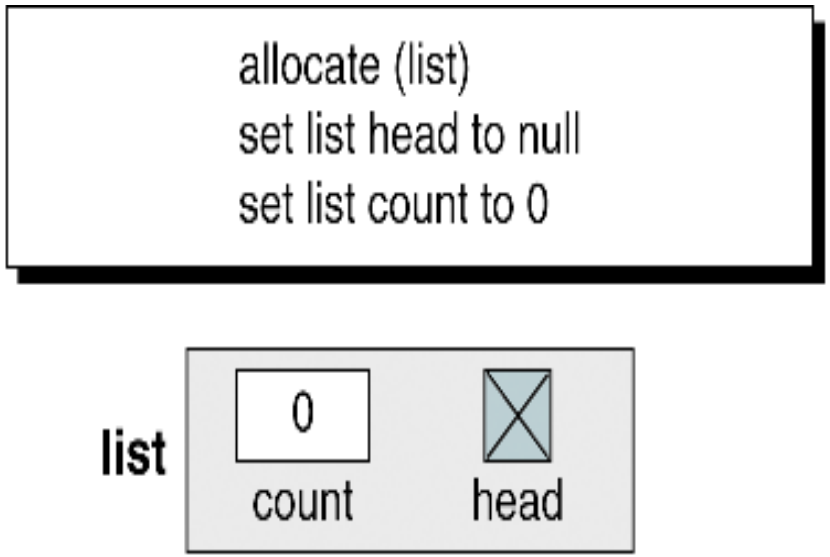


FIGURE 5-6 Create List

Create list

ALGORITHM 5-1 Create List

```
Algorithm createList (list)
Initializes metadata for list.
  Pre    list is metadata structure passed by reference
  Post   metadata initialized
1 allocate (list)
2 set list head  to null
3 set list count to 0
end createList
```

Insert node

- Only its logical predecessor is needed.
- There are three steps to the insertion:
  1. Allocate memory for the new node and move data to the node.
  2. Point the new node to its successor.
  3. Point the new node's predecessor to the new node.

Insert node

- Insert into empty list
- Insert at beginning
- Insert in middle
- Insert at end

Insert into empty list

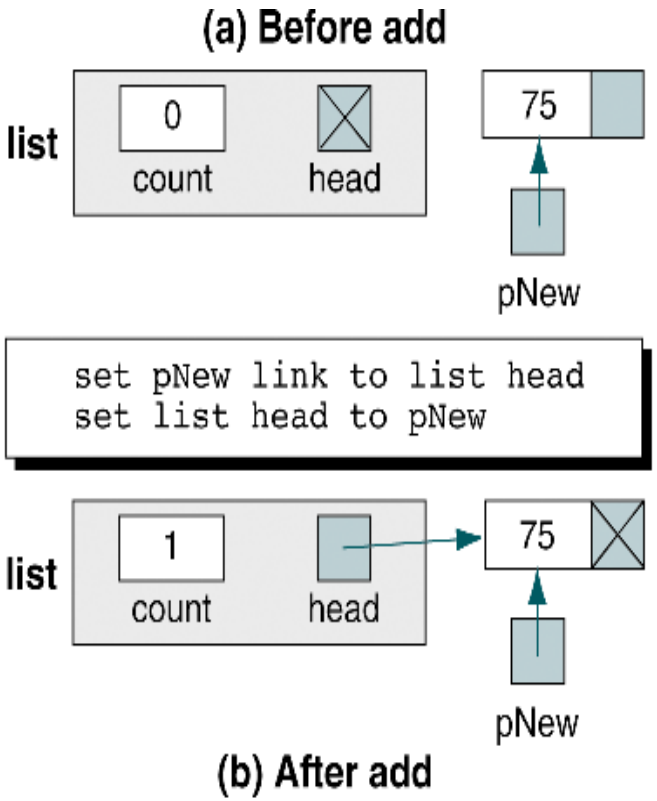


FIGURE 5-7 Add Node to Empty List

Insert at beginning

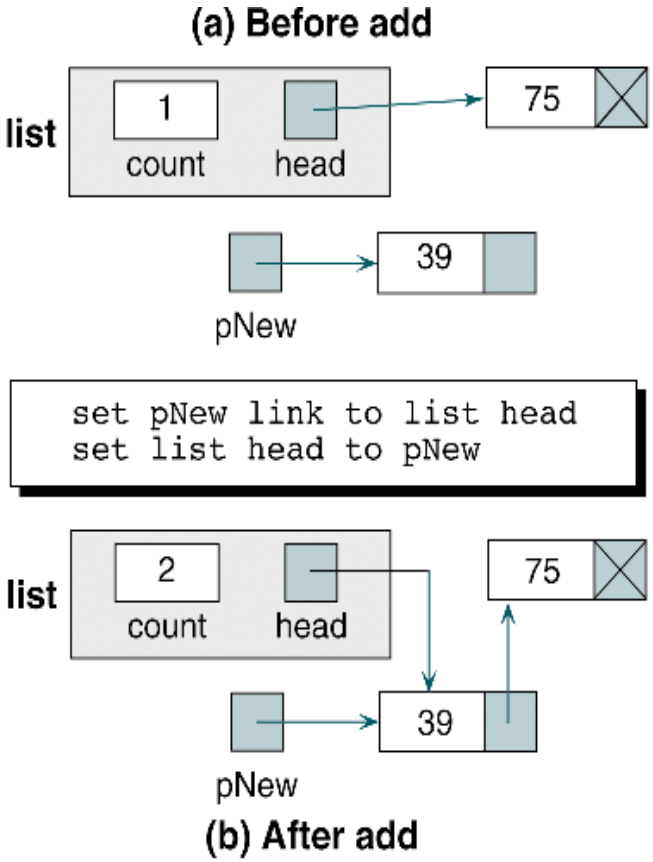


FIGURE 5-8 Add Node at Beginning

Insert in middle

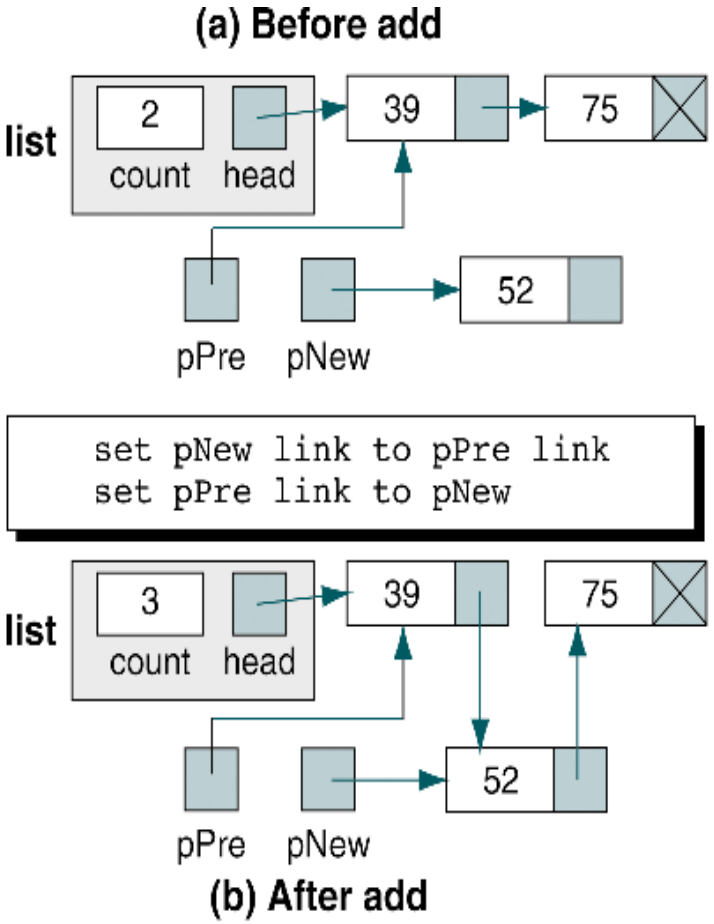


FIGURE 5-9 Add Node in Middle

Insert at end

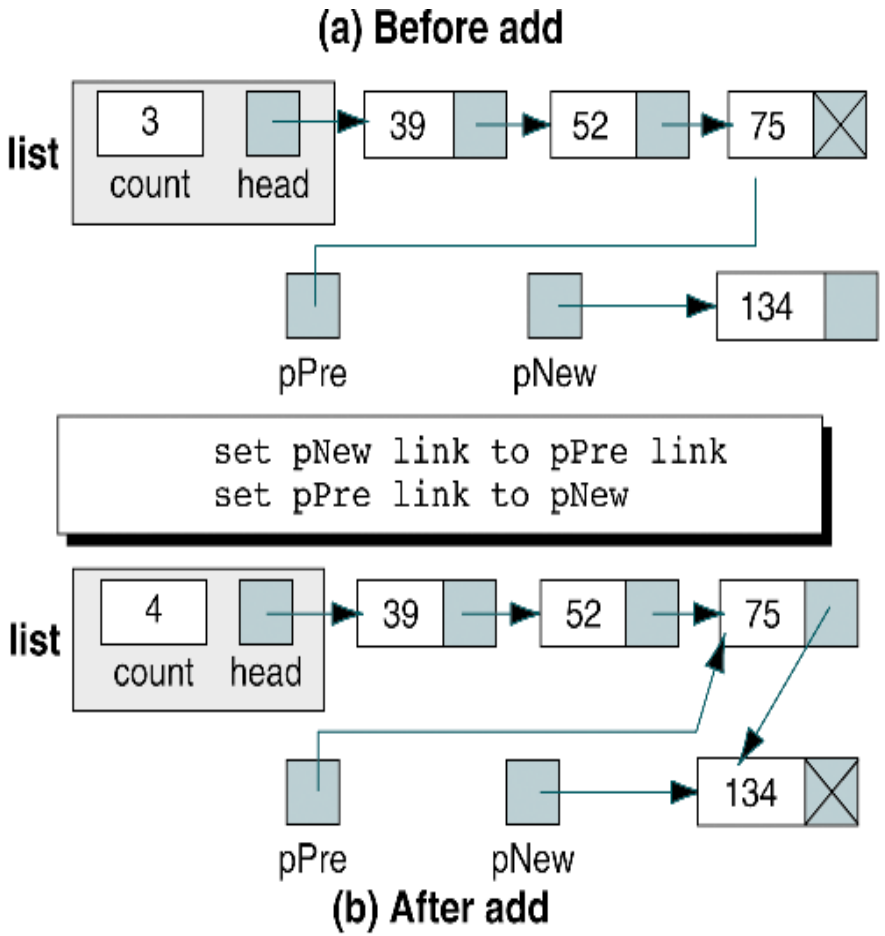


FIGURE 5-10 Add Node at End

ALGORITHM 5-2 Insert List Node

```
Algorithm insertNode (list, pPre, dataIn)
Inserts data into a new node in the list.
  Pre    list is metadata structure to a valid list
         pPre is pointer to data's logical predecessor
         dataIn contains data to be inserted
  Post   data have been inserted in sequence
  Return true if successful, false if memory overflow
1 allocate (pNew)
2 set pNew data to dataIn
3 if (pPre null)
  Adding before first node or to empty list.
  1  set pNew link to list head
  2  set list head to pNew
4 else
  Adding in middle or at end.
  1  set pNew link to pPre link
  2  set pPre link to pNew
5 end if
6 return true
end insertNode
```

Delete node

- Delete first node
- General delete case

Delete first node

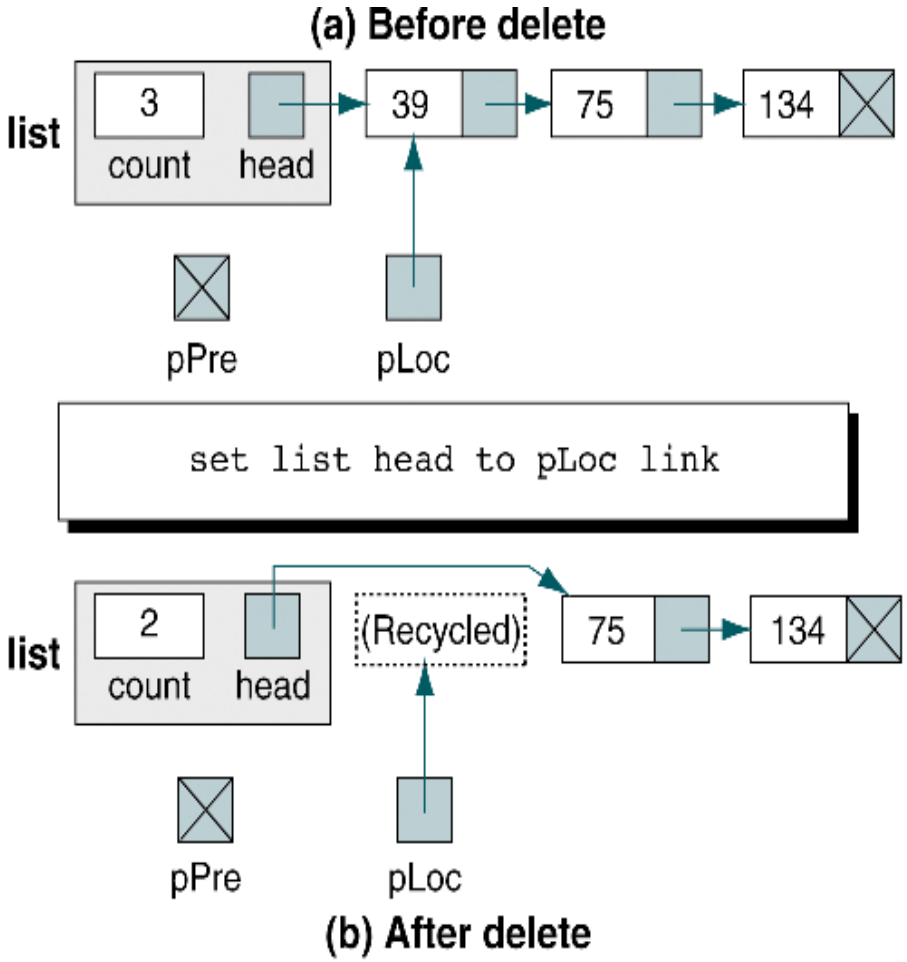


FIGURE 5-11 Delete First Node

Delete general case

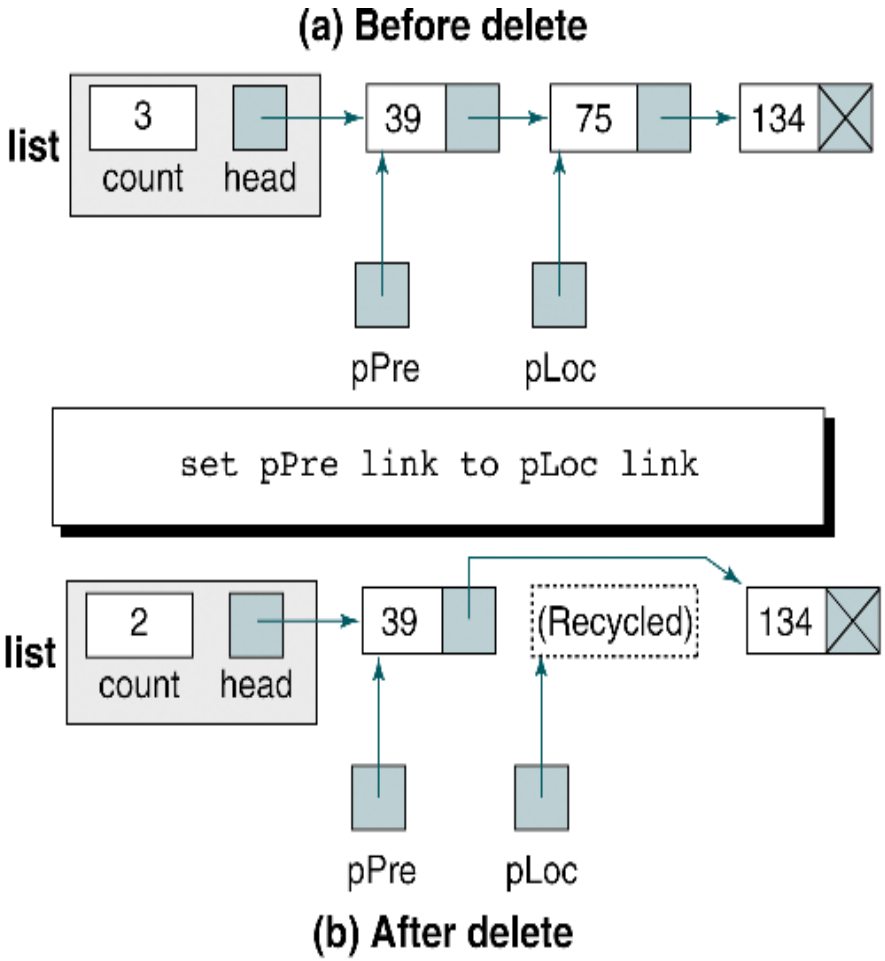


FIGURE 5-12 Delete General Case

ALGORITHM 5-3 List Delete Node

```
Algorithm deleteNode (list, pPre, pLoc, dataOut)
Deletes data from list & returns it to calling module.
  Pre  list is metadata structure to a valid list
       Pre is a pointer to predecessor node
       pLoc is a pointer to node to be deleted
       dataOut is variable to receive deleted data
  Post data have been deleted and returned to caller
1 move pLoc data to dataOut
2 if (pPre null)
  Deleting first node
  1  set list head to pLoc link
3 else
  Deleting other nodes
  1  set pPre link to pLoc link
4 end if
5 recycle (pLoc)
end deleteNode
```

List Search

• A list search is used by algorithms to locate data in a list.

- To insert data, we need to know the logical predecessor to the new data.

- To delete data, we need to know the node to be deleted and identify it's logical predecessor.

- To retrieve data, we need to search the list and find the data.

• Sequential search is used for list search because there is no physical relationship among the nodes; returns the locations of an element. If not found, returns the location of last element.

• Given a target key in the **ordered list**, if a node in the list matches the target value, the search returns true. If no key matches found, it returns false.

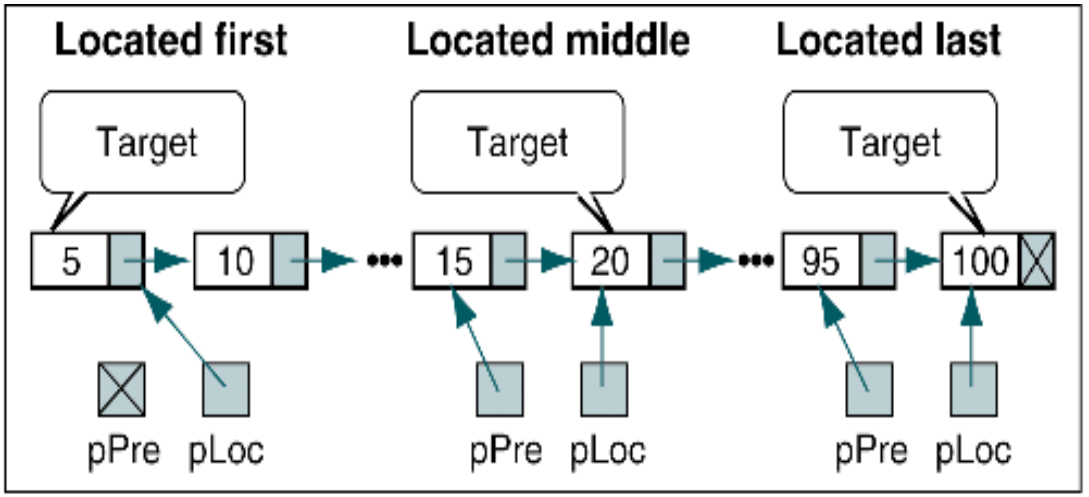
List search

```
Algorithm searchList (list, pPre, pLoc, target)
Searches list and passes back address of node containing
target and its logical predecessor.
  Pre   list is metadata structure to a valid list
        pPre is pointer variable for predecessor
        pLoc is pointer variable for current node
        target is the key being sought
  Post  pLoc points to first node with equal/greater key
        -or- null if target > key of last node
        pPre points to largest node smaller than key
        -or- null if target < key of first node
  Return true if found, false if not found
```

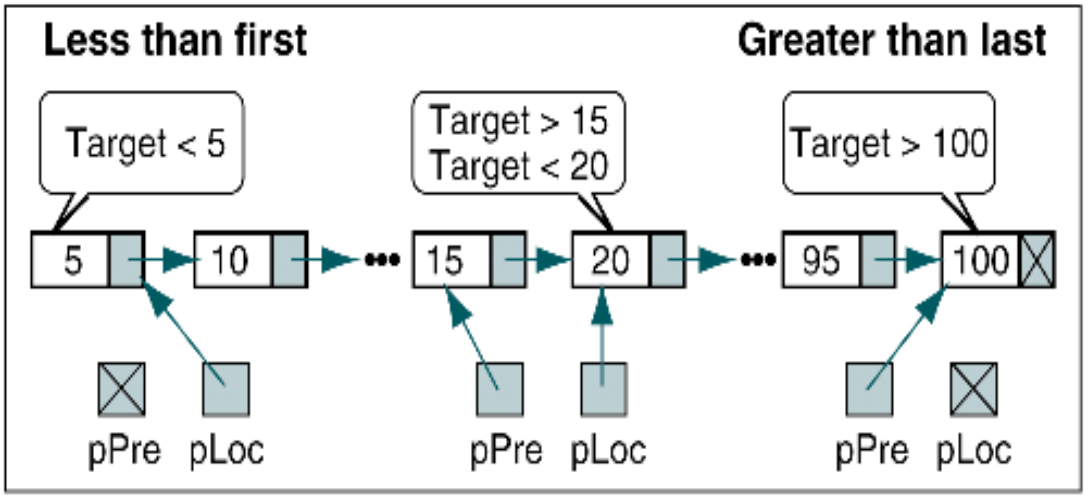
List search

Condition	pPre	pLoc	Return
Target < first node	Null	First node	False
Target = first node	Null	First node	True
First < target < last	Largest node < target	First node > target	False
Target = middle node	Node's predecessor	Equal node	True
Target = last node	Last's predecessor	Last node	True
Target > last node	Last node	Null	False

TABLE 5-1 List Search Results



(a) Successful searches (return true)



(b) Unsuccessful searches (return false)

FIGURE 5-13 Ordered List Search

ALGORITHM 5-4 Search List

```
Algorithm searchList (list, pPre, pLoc, target)
Searches list and passes back address of node containing
target and its logical predecessor.
  Pre   list is metadata structure to a valid list
        pPre is pointer variable for predecessor
        pLoc is pointer variable for current node
        target is the key being sought
  Post  pLoc points to first node with equal/greater key
        -or- null if target > key of last node
        pPre points to largest node smaller than key
        -or- null if target < key of first node
  Return true if found, false if not found
1 set pPre to null
2 set pLoc to list head
3 loop (pLoc not null AND target > pLoc key)
  1  set pPre to pLoc
  2  set pLoc to pLoc link
4 end loop
5 if (pLoc null)
  Set return value
  1  set found to false
```

continued

ALGORITHM 5-4 Search List *(continued)*

```
6 else
  1  if (target equal pLoc key)
     1  set found to true
  2  else
     1  set found to false
  3  end if
7 end if
8 return found
end searchList
```

Retrieve node

- Retrieve node uses search node to locate the data in the list.
- If the data are found, it moves the data to the output area in the calling module and returns true. If data are not found, returns false.

ALGORITHM 5-5 Retrieve List Node

```
Algorithm retrieveNode (list, key, dataOut)
Retrieves data from a list.
  Pre    list is metadata structure to a valid list
         key is target of data to be retrieved
         dataOut is variable to receive retrieved data
  Post   data placed in dataOut
          -or- error returned if not found
  Return true if successful, false if data not found
1 set found to searchList (list, pPre, pLoc, key)
2 if (found)
  1  move pLoc data to dataOut
3 end if
4 return found
end retrieveNode
```

Empty list

ALGORITHM 5-6 Empty List

```
Algorithm emtpyList (list)
Returns Boolean indicating whether the list is empty.
  Pre    list is metadata structure to a valid list
  Return true if list empty, false if list contains data
1 if (list count equal 0)
  1  return true
2 else
  1  return false
end emptyList
```

Full list

ALGORITHM 5-7 Full List

```
Algorithm fullList (list)
Returns Boolean indicating whether or not the list is full.
  Pre    list is metadata structure to a valid list
  Return false if room for new node; true if memory full
1 if (memory full)
  1  return true
2 else
  2  return false
3 end if
4 return true
end fullList
```

List count

ALGORITHM 5-8 List Count

```
Algorithm listCount (list)
Returns integer representing number of nodes in list.
  Pre    list is metadata structure to a valid list
  Return count for number of nodes in list
 1 return (list count)
end listCount
```

Traverse list

Traversal starts at the first node and examines each node in the succession until the last node has been processed.

Because we need to remember where we are in the list from one call to the next, we need to add a current position pointer to the head structure.

Each call needs to know whether it is starting from the beginning of the list or continuing from the last node processed. This information is communicated through the parameter list.

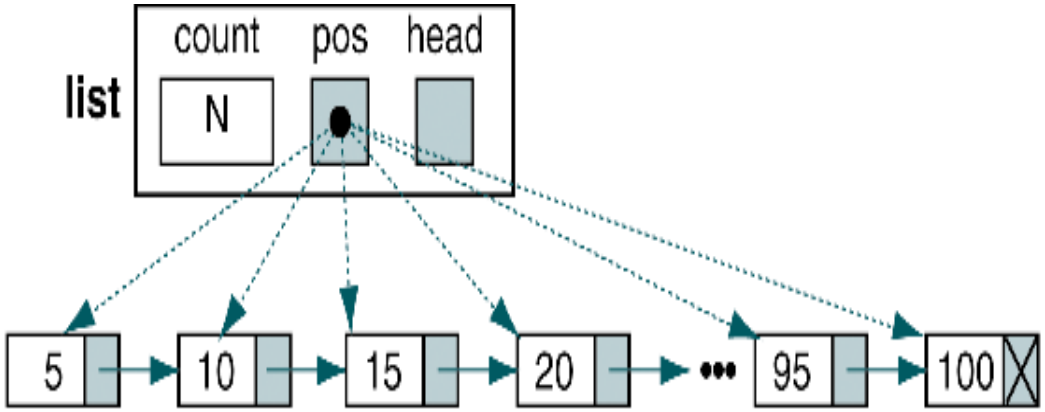


FIGURE 5-14 List Traversal

ALGORITHM 5-9 Traverse List

```
Algorithm getNext (list, fromWhere, dataOut)
Traverses a list. Each call returns the location of an
element in the list.
  Pre    list is metadata structure to a valid list
         fromWhere is 0 to start at the first element
         dataOut is reference to data variable
  Post   dataOut contains data and true returned
         -or- if end of list, returns false
```

continued

ALGORITHM 5-9 Traverse List *(continued)*

```
   Return true if next element located
          false if end of list
1  if (empty list)
   1  return false
2  if (fromWhere is beginning)
   Start from first
   1  set list pos to list head
   2  move current list data to dataOut
   3  return true
3  else
   Continue from pos
   1  if (end of list)
      End of List
      1  return false
    2 else
      1  set list pos to next node
      2  move current list data to dataOut
      3  return true
    3 end if
4  end if
end getNext
```

Destroy list

ALGORITHM 5-10 Destroy List

```
Algorithm destroyList (pList)
Deletes all data in list.
  Pre    list is metadata structure to a valid list
```

continued

```
  Post   All data deleted
1 loop (not at end of list)
  1  set list head to successor node
  2  release memory to heap
2 end loop
  No data left in list. Reset metadata.
3 set list pos to null
4 set list count to 0
end destroyList
```

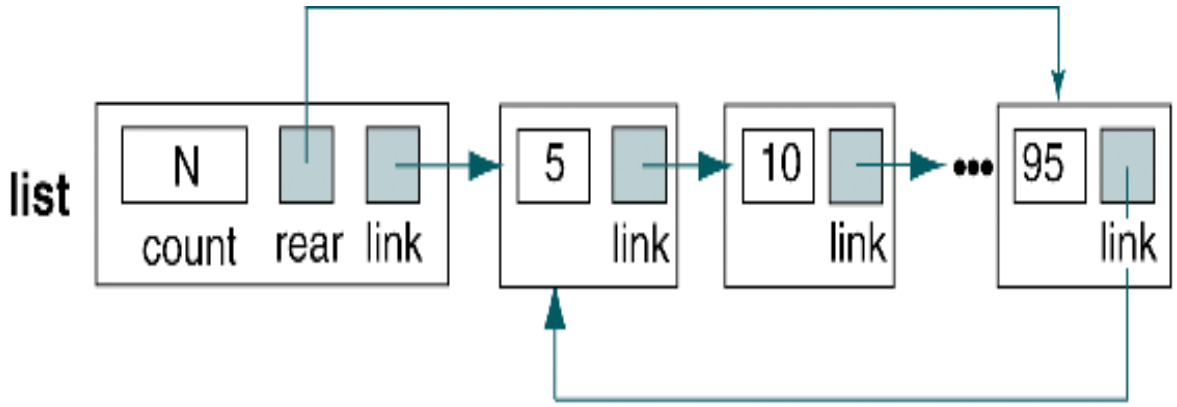


FIGURE 5-19 Circularly Linked List

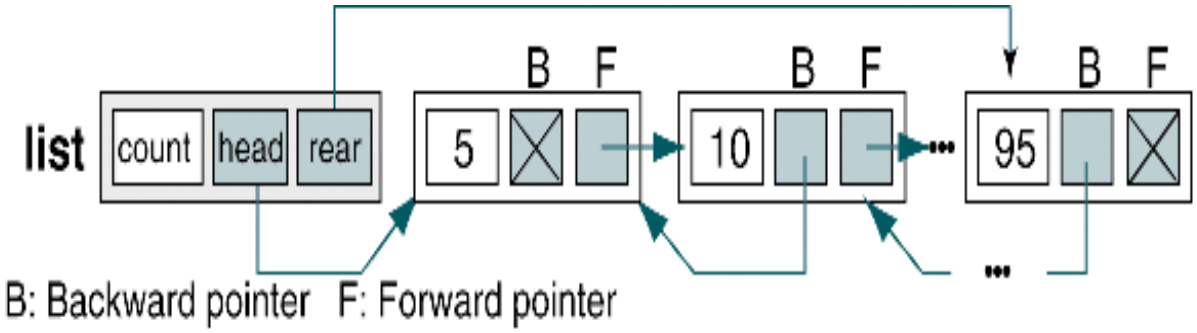
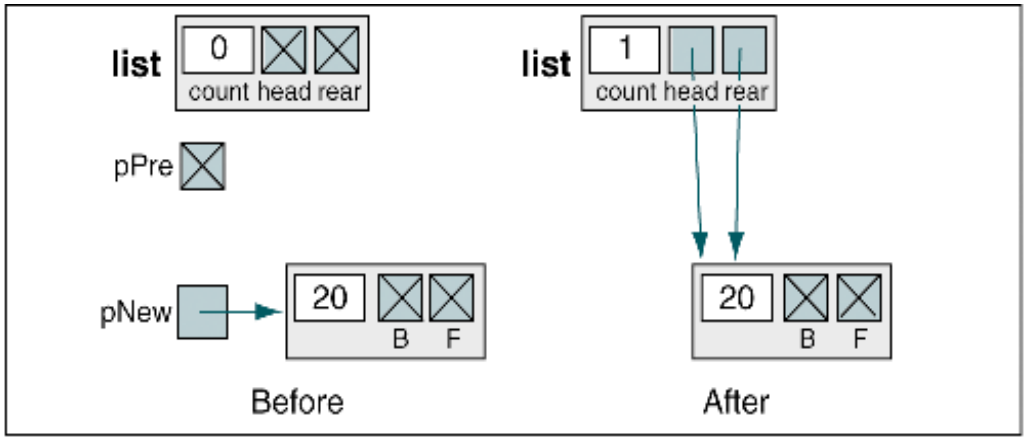
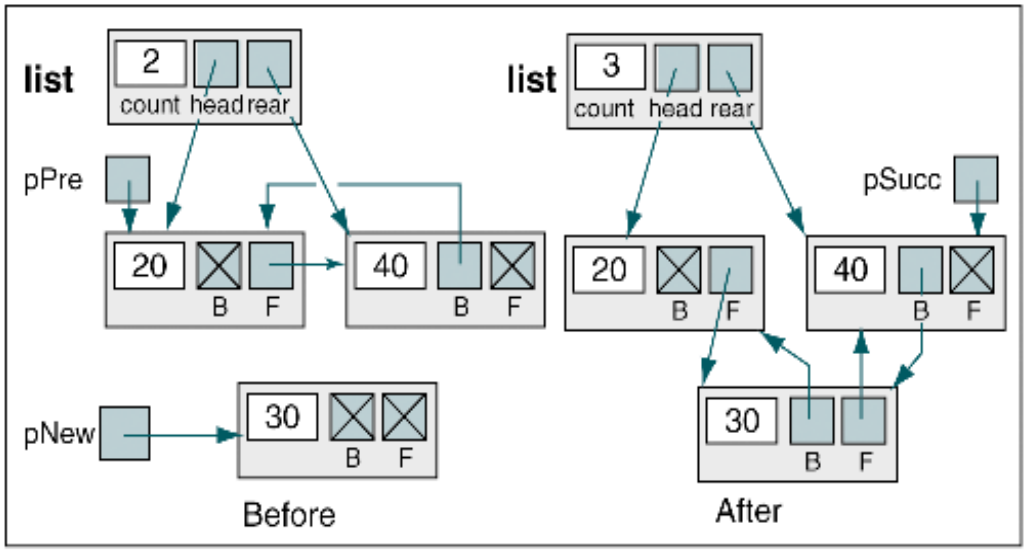


FIGURE 5-20 Doubly Linked List



(a) Insert into null list or before first node



(b) Insert between two nodes

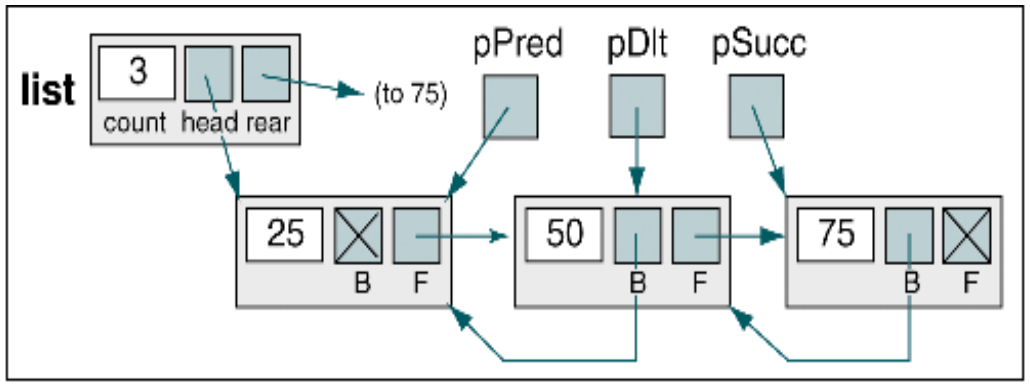
FIGURE 5-21 Doubly Linked List Insert

ALGORITHM 5-11 Doubly Linked List Insert

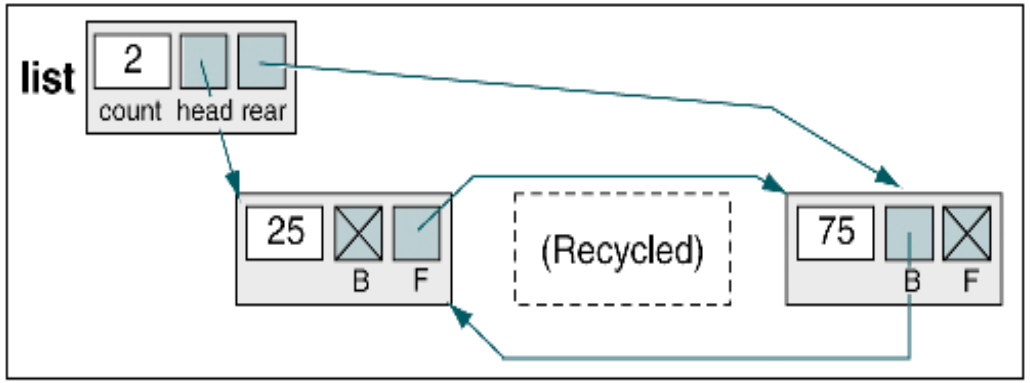
```
Algorithm insertDbl (list, dataIn)
This algorithm inserts data into a doubly linked list.
  Pre    list is metadata structure to a valid list
         dataIn contains the data to be inserted
  Post   The data have been inserted in sequence
  Return 0: failed--dynamic memory overflow
         1: successful
         2: failed--duplicate key presented
1 if (full list)
  1  return 0
2 end if
  Locate insertion point in list.
3 set found to searchList
       (list, predecessor, successor, dataIn key)
4 if (not found)
  1  allocate new node
  2  move dataIn to new node
  3  if (predecessor is null)
     Inserting before first node or into empty list
     1  set new node back pointer to null
     2  set new node fore pointer to list head
     3  set list head to new node
  4  else
       Inserting into middle or end of list
     1  set new node fore pointer to predecessor fore pointer
     2  set new node back pointer to predecessor
```

ALGORITHM 5-11 Doubly Linked List Insert (continued)

```
  5  end if
     Test for insert into null list or at end of list
  6  if (predecessor fore null)
       Inserting at end of list--set rear pointer
     1 set list rear to new node
  7  else
       Inserting in middle of list--point successor to new
     1 set successor back to new node
  8  end if
  9  set predecessor fore to new node
  10 return 1
 5 end if
   Duplicate data. Key already exists.
 6 return 2
end insertDbl
```



(a) Before delete



(b) After deleting 50

FIGURE 5-22 Doubly Linked List Delete

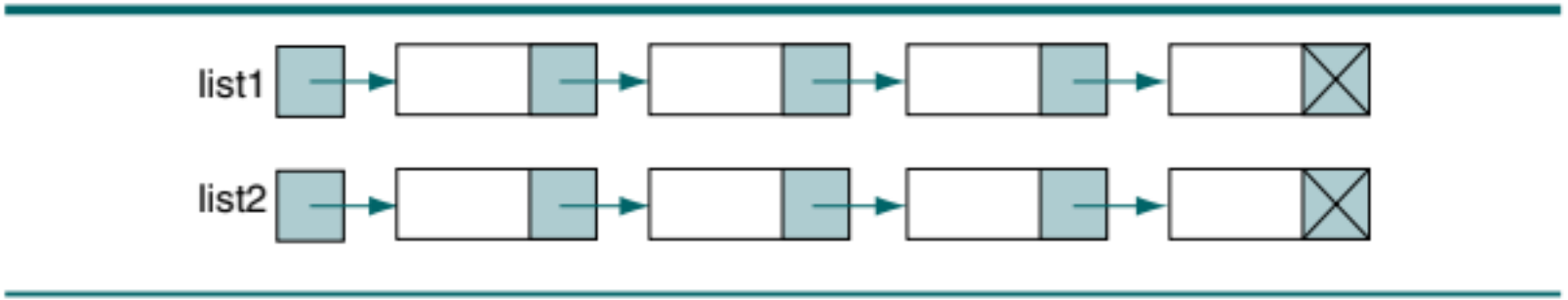
ALGORITHM 5-12 Doubly Linked List Delete

```
Algorithm deleteDbl (list, deleteNode)
This algorithm deletes a node from a doubly linked list.
   Pre    list is metadata structure to a valid list
          deleteNode is a pointer to the node to be deleted
   Post   node deleted
 1 if (deleteNode null)
   1  abort ("Impossible condition in delete double")
 2 end if
 3 if (deleteNode back not null)
   Point predecessor to successor
   1  set predecessor      to deleteNode back
   2  set predecessor fore to deleteNode fore
 4 else
   Update head pointer
   1  set list head to deleteNode fore
 5 end if
 6 if (deleteNode fore not null)
   Point successor to predecessor
   1  set successor      to deleteNode fore
   2  set successor back to deleteNode back
 7 else
   Point rear to predecessor
   1  set list rear to deleteNode back
 8 end if
 9 recycle (deleteNode)
end deleteDbl
```

Exercises

What would happen if we applied the following statements to the two lists

```
1 set temp to list1
2 loop (temp link not null)
  1 set temp to temp link
3 end loop
4 set temp link to list2
```



What would happen if we applied the following statements to this list?

```
1 set temp to pList
2 loop (temp link not null)
  1 set temp to temp link
3 end loop
4 set temp link to pList
```

